DAA — S1 Notes

Recurrence Analysis • Quick Sort • Merge Sort

1. Quick Sort — Best Case

Time complexity: O(n log n)

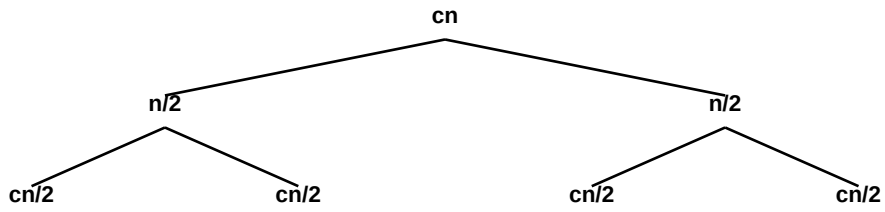
Number of subproblems: 2

Problem size: approximately half of the original size

Recurrence: T(n) = 2T(n/2) + cn

Recursion tree: Balanced tree

Result: T(n) = Θ(n log n)



balanced levels: log■ n

work per level ≈ n → Θ(n log n)

2. Merge Sort

Partition: find the middle element:

mid = ■(low + high) / 2■

The array is divided into two approximately equal subproblems.

Number of subproblems: 2

Recurrence: T(n) = 2T(n/2) + cn

Maximum scan/work at a level: n

Result: T(n) = Θ(n log n)

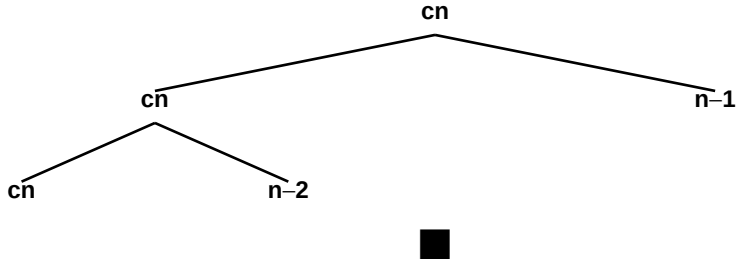
3. Quick Sort — Worst Case Partitioning

In the worst case, the pivot produces one empty subarray and one subarray of size n – 1.

T(n) = T(n – 1) + T(0) + Θ(n)

Since T(0) = Θ(1):

T(n) = T(n – 1) + cn



■

depth ≈ n → Θ(n²)

4. Worst-Case Recurrence — Iteration Method

Repeatedly substitute the recurrence until the base case is reached.

$$T(n) = T(n-1) + cn$$

$$= T(n-2) + c(n-1) + cn$$

$$= T(n-3) + c(n-2) + c(n-1) + cn$$

$$\blacksquare$$

$$= T(0) + c(1 + 2 + 3 + \blacksquare + n)$$

$$= T(0) + c \cdot n(n+1)/2$$

Therefore:

$$T(n) = \Theta(n^2)$$

5. Quick Sort — Average Case

Let $C_A(n)$ denote the average cost. If the pivot is at position k, the two subproblems have sizes k – 1 and n – k.

$$C_A(n) = n + 1 + (1/n) \sum_{1 \le k \le n} [C_A(k-1) + C_A(n-k)]$$

Using symmetry of the two sums:

$$C_A(n) = n + 1 + (2/n) \sum_{j=0}^{n-1} C_A(j)$$

Multiply by n:

$$nC_A(n) = n(n+1) + 2[C_A(0) + C_A(1) + \blacksquare + C_A(n-1)]$$

Replace n by n – 1:

$$(n-1)C_A(n-1) = n(n-1) + 2[C_A(0) + \blacksquare + C_A(n-2)]$$

Subtract the two equations:

$$nC_A(n) - (n-1)C_A(n-1) = 2n + 2C_A(n-1)$$

Hence:

$$nC_A(n) = 2n + (n+1)C_A(n-1)$$

Divide by n(n + 1):

$$C_A(n)/(n+1) = 2/(n+1) + C_A(n-1)/n$$

6. Average-Case Iteration

Starting from:

$C_A(n)/(n + 1) = C_A(n – 1)/n + 2/(n + 1)$

Iterate:

$= C_A(n – 2)/(n – 1) + 2/n + 2/(n + 1)$

$= C_A(n – 3)/(n – 2) + 2/(n – 1) + 2/n + 2/(n + 1)$

■

$= C_A(2)/3 + 2(1/4 + 1/5 + ■ + 1/(n + 1))$

Thus the summation grows logarithmically:

Average-case result:

$C_A(n) = \Theta(n \log n)$

7. Iteration Method — Standard Examples

Example 1

$T(n) = T(n/2) + 1$

$= T(n/4) + 1 + 1$

$= T(n/8) + 1 + 1 + 1$

■

$= T(n/2^k) + k$

At the base case: $n/2^k = 1 \Rightarrow k = \log_2 n$

$\therefore T(n) = T(1) + \log_2 n = \Theta(\log n)$

Example 2

$T(n) = T(n – 1) + 1$

$= T(n – 2) + 2 = ■ = T(1) + (n – 1)$

$\therefore T(n) = \Theta(n)$

Example 3

$T(n) = T(n – 1) + n$

$= T(n – 2) + (n – 1) + n$

■

$= T(0) + 1 + 2 + ■ + n$

$= T(0) + n(n + 1)/2$

$\therefore T(n) = \Theta(n^2)$

Example 4

$T(n) = T(n/2) + n^2$

$= T(n/4) + (n/2)^2 + n^2$

$= T(n/8) + (n/4)^2 + (n/2)^2 + n^2$

■

$= T(n/2^k) + n^2[1 + 1/4 + 1/16 + ■ + 1/4^{k–1}]$

With $k = \log_2 n$, the geometric series is bounded by a constant.

$\therefore T(n) = \Theta(n^2)$

Quick Reference

Topic	Recurrence / Result
Binary Search	$\Theta(\log n)$
Merge Sort	$T(n) = 2T(n/2) + cn \rightarrow \Theta(n \log n)$
Quick Sort — Best	$T(n) = 2T(n/2) + cn \rightarrow \Theta(n \log n)$
Quick Sort — Average	$C_A(n) \rightarrow \Theta(n \log n)$
Quick Sort — Worst	$T(n) = T(n - 1) + cn \rightarrow \Theta(n^2)$